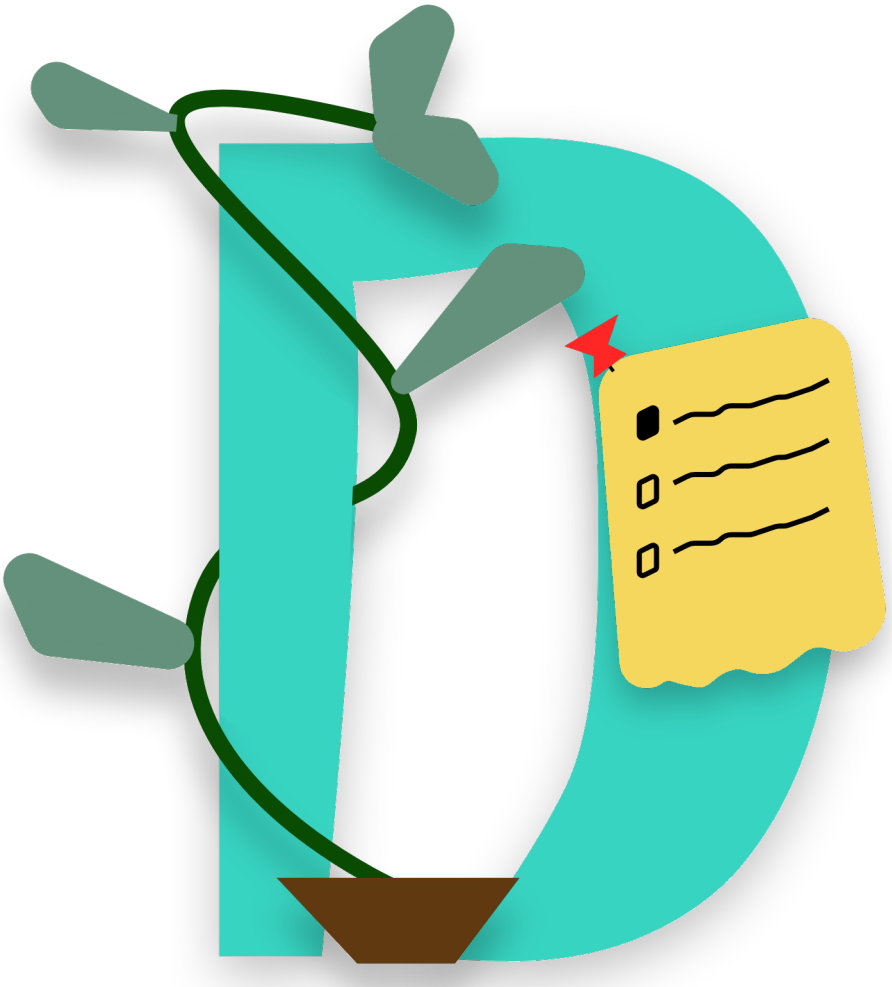


Diab -- Make a garden through your journey

Problem Statement

Having diabetes is a big challenge that requires great care for it to not go out of hand and cause you harm.

The main problem that people with diabetes face is a lack of motivation to keep track of all the things they need to sustain a healthy lifestyle.

Solution Statement

DIAB, is a gamified health tracking app that helps people who suffer from diabetes to keep track of all the information they require while receiving motivation through the game rewards, DIAB is meant to help track the following

Tracked Metrics / Tests

- Capillary Glucose (Fasting + 2h after each meal)
- Foot Care (2 times / day)
- Sugar Eaten Today
- Water Drank Today

- Minutes Walked
- Sleep Time
- Sleep Duration
- Venous Glucose (1 / 3 months)
- Hba1c Test (1 / 3 months)
- General Test (1 / 6 months)
- Ophthalmologist Consultation (yearly)
- Cardiologist Consultation (yearly)

User persona

Bouzidi Mounir

> Suffers from Type 2 Diabetes, 38 years old

Goals

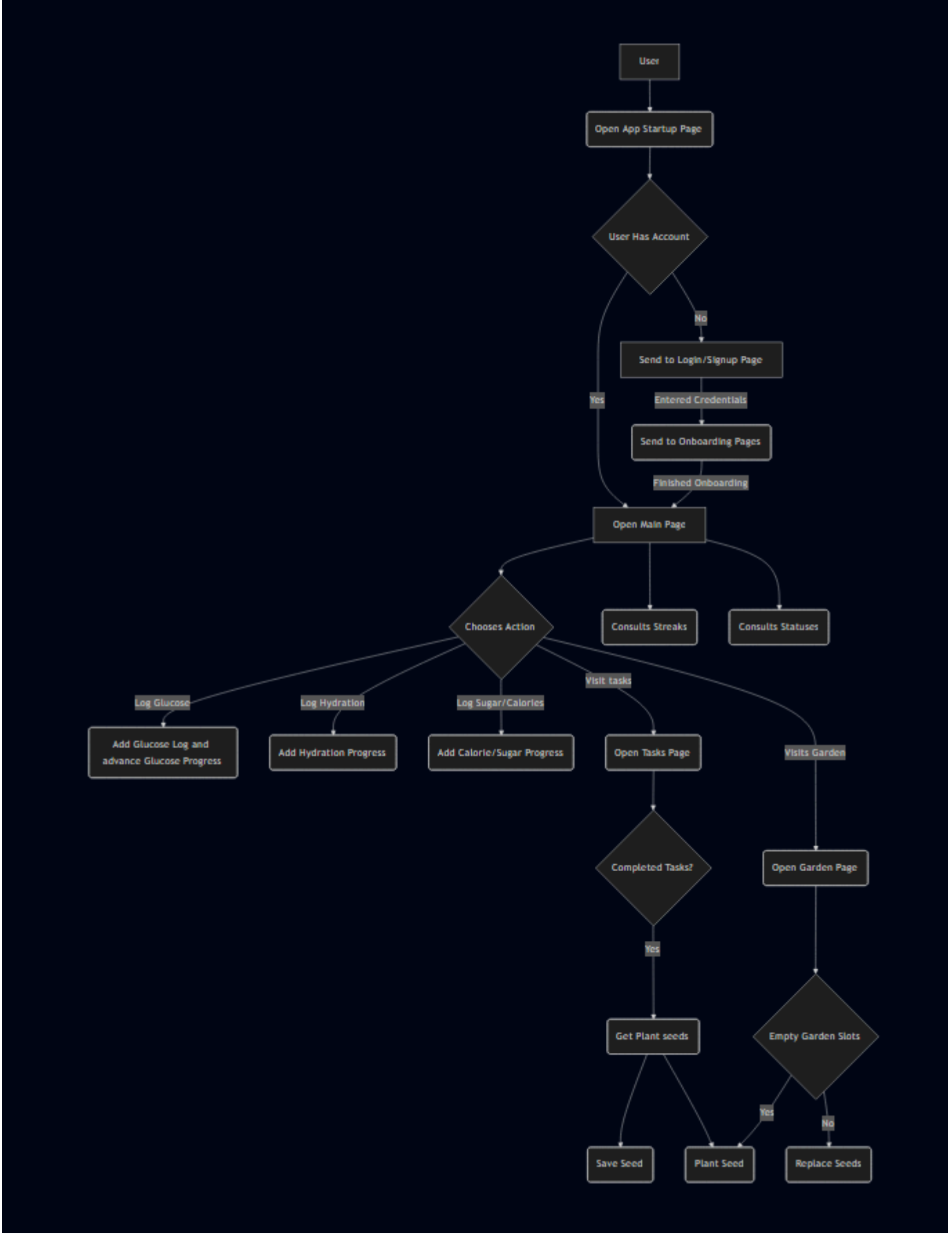
Bouzidi has many goals he wants to achieve in regards to his health:

- Keep his nutrition in check
- Keep track of all the monthly and yearly appointments

Pain points

- Suffers from the lack of a centralized place to keep track of his needs
- Lack of motivation due to not keeping track of his progress
- Always resets his progress back to zero inducing a negative feedback loop

User flow



Gamification Logic

The Game works by building a garden of plants that represent your achievements across different fields, so each time you achieve a good outcome in one of the sides of diabetes (diet, exercise, glucose, medicine) you get rewarded with some plant seeds to either plant or upgrade your plants so they grow prettier and stronger as you grow healthier.

This system also introduces plants based on streaks or compound achievements helping the user feel motivated to keep their entire life balanced not just one part.

The Following Are some example plants and achievements:

Plant	Achievement
Aloe Vera	Obtained By keeping a balanced Glucose for a day
Weeping Bonsai	Obtained By staying Hydrated for 8 days straight
Perfect Daisy	Obtained By Having all your stats to be perfect
Carrion Flower	Obtained By Having a 1 year streak
Olive Bonsai	Obtained By Rebalancing Your glucose after a bad day
Lavender Field	Obtained By running atleast 30mins for a week straight
Garlic	Obtained By visiting the cardiologist once
Ice plant	Obtained By Completing 10 visits on time

And Many More, users might be able to visit each others gardens to see their proudest achievements.

> Note that this system will always rely on the person being responsible and not just cheating their way, since there is no benefit to their cheating its up to them to keep true to themselves.
>
> This is due to most metrics are being tracked through pure user input

Figma Link

Link to the Figma file:
Click Here

Technical Specification

The Following Will be The technical Documentation for the Diab Implementation

Scope & Overview

Diab Is meant to be primarily used as a mobile app to track people progress in their diabetes journey.

Thus it will be created to run on android and iOS phones.

We'll be using a monolithic design with one centeral server that can handle our apps logic, we avoid microservices in this case as it introduces unnecessary complexity.

Tech Stack

Frontend

For the frontend we'll be using React-Native with the Expo Framework.

Other libraries We'll use in frontend include:

- `glue-stack ui`: a ui library similar to radix ui, used to quicken ui implementations and to not think much about common ui code.
- `NativeWind`: to enable tailwindcss like styling in our app.
- `Axios`: To handle HTTP Requests.

> The choices for these are stemming from the familiarity of the team with the web based equivalents.

Backend

For the Backend we'll use the Django Web framework, which enables fast development that's production ready for most applications.

- Framework: Django (Python)
- ORM: Django ORM
- Database: PostgreSQL or MySQL
- Authentication: Django Auth

> Any other libraries that might be used later on will be appended

Database Schema

Since we are using an SQL Based database we must have a clear database schema

Tables

Garden Tables

Table Name	Purpose	Relations
User	Stores user account information	–
Garden	Represents a user's garden instance	1:1 with User
PlantDefinition	Stores all possible plant types and their unlock conditions	1:n with GardenPlant
GardenPlant	Represents a plant owned by a specific garden (level, growth state)	m:1 with Garden, m:1 with PlantDefinition

Record Table

Table Name	Purpose	Relations
GlucoseRecord	Stores capillary glucose readings (fasting or post-meal)	m:1 with User
WaterRecord	Tracks water intake entries	m:1 with User
SleepRecord	Tracks sleep sessions including duration	m:1 with User
DietRecord	Tracks macronutrient intake (protein, sugar, fats)	m:1 with User
ActivityRecord	Tracks physical activity sessions and intensity	m:1 with User

Tasks Table

Table Name	Purpose	Relations
TaskDefinition	Stores all possible tasks users can complete	1:n with UserTaskCompletion
UserTaskCompletion	Records when a user completes a task and its associated value	m:1 with User, m:1 with TaskDefinition

Achievements Table

Table Name	Purpose	Relations
AchievementDefinition	Stores all possible achievements and their unlock conditions	1:n with UserAchievement
UserAchievement	Records achievements unlocked by users	m:1 with User, m:1 with AchievementDefinition

Streaks Table

Table Name	Purpose	Relations
UserStreak	Tracks current and best streaks for different categories	m:1 with User

Consultation Table

Table Name	Purpose	Relations
Consultations	Tracks scheduled and attended medical consultations	m:1 with User

Schema

```
User(
    id,
```

```
    email,
    password_hash,
    date_joined
)

Garden(
    id,
    user_id,
    created_at
)

PlantDefinition(
    id,
    name,
    description,
    unlock_condition_type,
    unlock_condition_value,
    rarity
)

GardenPlant(
    id,
    garden_id,
    plant_id,
    level,
    planted_at,
    last_upgraded_at,
)

/* Tracking System Tables */

GlucoseRecord(
    id,
    user_id,
    value,
    unit,
    is_fasting,
    meal,
    recorded_at
)

WaterRecord(
    id,
    user_id,
    amount_ml,
    recorded_at
)

SleepRecord(
    id,
    user_id,
    start_time,
    end_time,
```

```
    duration_minutes
)

DietRecord(
    id,
    user_id,
    eaten_at,
    protein,
    sugar,
    fats
)

ActivityRecord(
    id,
    user_id,
    start_time,
    end_time,
    exercise_type,
    exercise_intensity
)



/* Tasks */

TaskDefinition(
    id,
    name,
    description,
    category,
    evaluation_logic_type,
    reward_points,
)


UserTaskCompletion(
    id,
    user_id,
    task_id,
    completed_at,
    value_recorded
)

/*Achievements*/

AchievementDefinition(
    id,
    name,
    description,
    condition_type,
    condition_value,
    plant_reward_id
)
```

```
UserAchievement(
    id,
    user_id,
    achievement_id,
    unlocked_at
)


/* Streaks */

UserStreak(
    user_id,
    streak_type,
    curremt_count,
    best_count,
    last_updated
)


/* Consultations */

Consultations(
    id,
    user_id,
    scheduled_at,
    consultation_type,
    attended
)
```

API

This will be a definition of the API exposed from the backend to the frontend.

> Note that for now I have not defined the request or reponse payloads, just a quick blueprint, Ill extend this later

authentication

```
POST   /api/auth/register/
```

- registers a new user
- req
- *res*

```
POST   /api/auth/login/
```

- logs in a user

- req
- *res*

```
POST   /api/auth/logout/
```

- Logs out a user
- req
- *res*

```
GET    /api/auth/me/
```

- Returns information of the logged in user
- req
- *res*

garden

```
GET    /api/garden/
```

- Get information about the garden
- req
- *res*

```
GET    /api/garden/plants/
```

- Get information about the planted and non-planted plants in the garden
- req
- *res*

```
POST   /api/garden/plants/
```

- Add a plant to the garden
- req
- *res*

```
PATCH  /api/garden/plants/{id}/upgrade/
```

- Update existing plant
- req
- *res*

records

> These are obvious, the get requests return a list of records enough to plot a week and the post requests add a new record. the request bodies are always the entire table row (minus the ids) and responses are an array of the table rows (minus the ids).

Glucose

```
POST   /api/records/glucose/
```

```
GET    /api/records/glucose/
```

Water

```
POST   /api/records/water/
```

```
GET    /api/records/water/
```

Diet

```
POST   /api/records/diet/
```

```
GET    /api/records/diet/
```

Sleep

```
POST   /api/records/sleep/
```

```
GET    /api/records/sleep/
```

Activity

```
POST   /api/records/activity/
```

```
GET    /api/records/activity/
```

tasks

```
GET    /api/tasks/
```

- Get tasks for the user

- req
- *res*

```
POST   /api/tasks/complete/
```

- Mark a task as complete
- req
- *res*

```
GET    /api/tasks/completions/
```

- Get all completed tasks of the user
- req
- *res*

achievements

```
GET    /api/achievements/
```

- Get all achievements
- req
- *res*

```
GET    /api/achievements/unlocked/
```

- Get list of unlocked achievements
- req
- *res*

consultations

```
GET    /api/streaks/
```

- Get All current streak statuses
- req
- *res*

Consultation

```
GET    /api/consultations/
```

- Get all current consulations for the user
- req

- *res*

```
POST   /api/consultations/
```

- Add a new consultation
- req
- *res*

```
PATCH  /api/consultations/{id}/
```

- Update a given consultation
- req
- *res*

Tasks and achievement systems